PRACTICAL-4

Aim: To develop a game-playing AI agent for Tic-Tac-Toe using the Minimax algorithm and improve its performance using Alpha-Beta Pruning.

PROGRAM:

```
# Tic-Tac-Toe using Minimax and Alpha-Beta Pruning

board = [" "] * 9

def show(board):
    print()
    for row_start in [0, 3, 6]:
        a = board[row_start]
        b = board[row_start + 1]
        c = board[row_start + 2]

        print(a + "|" + b + "|" + c)

        if row_start < 6:
            print("-+-+-")
    print()

LINES = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8],
    [0, 3, 6], [1, 4, 7], [2, 5, 8],
    [0, 4, 8], [2, 4, 6]
]

def winner(board):
    for a, b, c in LINES:
        if board[a] != " " and board[a] == board[b] and board[b] == board[c]:
            return board[a]
    return None

def empty_boxes(board):
    free = []

    for i in range(9):
        if board[i] == " ":
            free.append(i)

    return free

# ---------- MINIMAX ----------
```

```
def minimax(board, turn):

    win = winner(board)

    if win == "X":
        return 1

    if win == "O":
        return -1

    if len(empty_boxes(board)) == 0:
        return 0

    if turn == "X":
        best = -2

        for box in empty_boxes(board):
            board[box] = "X"

            score = minimax(board, "O")

            board[box] = " "

            if score > best:
                best = score

        return best

    else:
        best = 2

        for box in empty_boxes(board):
            board[box] = "O"

            score = minimax(board, "X")

            board[box] = " "

            if score < best:
                best = score

        return best

def best_move(board):
```

```
    best_score = -2
    best_box = empty_boxes(board)[0]

    for box in empty_boxes(board):

        board[box] = "X"

        score = minimax(board, "O")

        board[box] = " "

        if score > best_score:
            best_score = score
            best_box = box

    return best_box

print("The AI can now look ahead and choose its best move.")

# Simple opponent
def simple_opponent(board):
    return empty_boxes(board)[0]

# ---------------- PLAY MINIMAX GAME ----------------

board = [" "] * 9
turn = "X"

print("The game begins!")
print("X is our smart AI.")
print("O always takes the first free box.")

show(board)

while winner(board) is None and len(empty_boxes(board)) > 0:

    if turn == "X":

        move = best_move(board)
        board[move] = "X"

        print("AI (X) plays box", move)

        turn = "O"

    else:
```

```
        move = simple_opponent(board)
        board[move] = "O"

        print("Player (O) plays box", move)

        turn = "X"

    show(board)

who = winner(board)

if who == "X":
    print("Result: The AI (X) WINS!")

elif who == "O":
    print("Result: O wins.")

else:
    print("Result: It is a draw.")

# ---------- TEST AGAINST RANDOM PLAYER ----------

import random

wins = 0
draws = 0
losses = 0

for game_number in range(30):

    board = [" "] * 9
    turn = "X"

    while winner(board) is None and len(empty_boxes(board)) > 0:

        if turn == "X":

            board[best_move(board)] = "X"
            turn = "O"

        else:

            board[random.choice(empty_boxes(board))] = "O"
            turn = "X"
```

```
    result = winner(board)

    if result == "X":
        wins = wins + 1

    elif result == "O":
        losses = losses + 1

    else:
        draws = draws + 1

print("Out of 30 games against a random player:")
print("AI wins :", wins)
print("Draws   :", draws)
print("AI loses:", losses)
print()

print("Wins and draws can change each run,")
print("since the opponent moves randomly.")
print("But the losses should always stay at zero.")

# ---------- ALPHA-BETA PRUNING ----------

def alphabeta(board, turn, alpha, beta):

    win = winner(board)

    if win == "X":
        return 1

    if win == "O":
        return -1

    if len(empty_boxes(board)) == 0:
        return 0

    if turn == "X":

        best = -2

        for box in empty_boxes(board):

            board[box] = "X"

            score = alphabeta(board, "O", alpha, beta)
```

```
            board[box] = " "

            if score > best:
                best = score

            if best > alpha:
                alpha = best

            if alpha >= beta:
                break

        return best

    else:

        best = 2

        for box in empty_boxes(board):

            board[box] = "O"

            score = alphabeta(board, "X", alpha, beta)

            board[box] = " "

            if score < best:
                best = score

            if best < beta:
                beta = best

            if alpha >= beta:
                break

        return best

print("The AI now knows the faster Alpha-Beta way.")

def best_move_ab(board):

    best_score = -2
    best_box = empty_boxes(board)[0]

    for box in empty_boxes(board):

        board[box] = "X"
```

```
        score = alphabeta(board, "O", -2, 2)

        board[box] = " "

        if score > best_score:
            best_score = score
            best_box = box

    return best_box

# ---------------- PLAY ALPHA-BETA GAME ----------------

board = [" "] * 9
turn = "X"

print("Same game, this time the AI uses Alpha-Beta to choose its move.")

show(board)

while winner(board) is None and len(empty_boxes(board)) > 0:

    if turn == "X":

        move = best_move_ab(board)
        board[move] = "X"

        print("AI (X) plays box", move)

        turn = "O"

    else:

        move = simple_opponent(board)
        board[move] = "O"

        print("Player (O) plays box", move)

        turn = "X"

    show(board)

who = winner(board)

if who == "X":
    print("Result: The AI (X) WINS!")
```

```
elif who == "O":
    print("Result: O wins.")

else:
    print("Result: It is a draw.")
```

OUTPUT:

```
The AI can now look ahead and choose its best move.
The game begins!
X is our smart AI.
O always takes the first free box.

 | |
-+-+-
 | |
-+-+-
 | |

AI (X) plays box 0

X| |
-+-+-
 | |
-+-+-
 | |

Player (O) plays box 1

X|O|
-+-+-
 | |
-+-+-
 | |

AI (X) plays box 3

X|O|
-+-+-
X| |
-+-+-
 | |

Player (O) plays box 2
```

```
X|O|O
-+-+-
X| |
-+-+-
 | |
```

AI (X) plays box 4

```
X|O|O
-+-+-
X|X|
-+-+-
 | |
```

Player (O) plays box 5

```
X|O|O
-+-+-
X|X|O
-+-+-
 | |
```

AI (X) plays box 6

```
X|O|O
-+-+-
X|X|O
-+-+-
X| |
```

Result: The AI (X) WINS!
Out of 30 games against a random player:
AI wins : 30
Draws : 0
AI loses: 0

Wins and draws can change each run,
since the opponent moves randomly.
But the losses should always stay at zero.
The AI now knows the faster Alpha-Beta way.
Same game, this time the AI uses Alpha-Beta to choose its move.

```
 | |
-+-+-
 | |
```

```
-+-+-
 | |
```

AI (X) plays box 0

```
X| |
-+-+-
 | |
-+-+-
 | |
```

Player (O) plays box 1

```
X|O|
-+-+-
 | |
-+-+-
 | |
```

AI (X) plays box 3

```
X|O|
-+-+-
X| |
-+-+-
 | |
```

Player (O) plays box 2

```
X|O|O
-+-+-
X| |
-+-+-
 | |
```

AI (X) plays box 4

```
X|O|O
-+-+-
X|X|
-+-+-
 | |
```

Player (O) plays box 5

```
X|O|O
```

```
-+-+-
X|X|O
-+-+-
 | |
```

AI (X) plays box 6

```
X|O|O
-+-+-
X|X|O
-+-+-
X| |
```

Result: The AI (X) WINS!